

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant est capable de :

- Définir et appeler des fonctions en Python
- Utiliser des paramètres et des valeurs de retour
- Comprendre la portée des variables (locale vs globale)
- Structurer un programme en fonctions réutilisables
- Organiser des fonctions dans un module Python

Plan du TP

Phase	Contenu	Durée
Accueil	Rappel sur les fonctions	10 min
Exercice 1	Fonction calculer_moyenne	15 min
Exercice 2	Fonctions max/min	15 min
Exercice 3	Fonction palindrome	20 min
Exercice 4	Générateur d'exercices	20 min
Exercice 5	Module calculs statistiques	10 min

Prérequis

- TP 01-03 bouclés (variables, conditions, boucles)
- Notion de fonction vue en cours (def, return)

Exercice 1 — Fonction calculer_moyenne (15 min)

Énoncé : Créer un script `moyenne_fonction.py` qui définit une fonction `calculer_moyenne(liste)` qui prend une liste de nombres et retourne leur moyenne. Le programme principal demande 5 notes à l'utilisateur, appelle la fonction et affiche le résultat.

Exemple attendu :

```
Note 1 : 12
Note 2 : 15
Note 3 : 10
Note 4 : 18
Note 5 : 14
```

Moyenne : 13.80/20

Corrigé :

```
def calculer_moyenne(liste):
    """Retourne la moyenne des éléments d'une liste."""
    if len(liste) == 0:
        return 0
    return sum(liste) / len(liste)

notes = []
for i in range(5):
    note = float(input(f"Note {i + 1} : "))
    notes.append(note)

moyenne = calculer_moyenne(notes)
print(f"Moyenne : {moyenne:.2f}/20")
```

Exercice 2 — Fonctions max/min personnalisées (15 min)

Énoncé : Créer un script `minmax.py` qui définit (sans utiliser `max()`/`min()`) :

1. `trouver_max(liste)` — retourne la valeur maximale
2. `trouver_min(liste)` — retourne la valeur minimale
3. `trouver_extremes(liste)` — retourne (min, max) en une seule fonction

Le programme demande 4 nombres et affiche min, max et l'étendue.

Corrigé :

```
def trouver_max(liste):
    max_val = liste[0]
    for val in liste:
        if val > max_val:
            max_val = val
    return max_val

def trouver_min(liste):
    min_val = liste[0]
    for val in liste:
        if val < min_val:
            min_val = val
    return min_val

def trouver_extremes(liste):
    return trouver_min(liste), trouver_max(liste)

nombres = []
for i in range(4):
    n = float(input(f"Nombre {i + 1} : "))
    nombres.append(n)

min_val, max_val = trouver_extremes(nombres)
etendue = max_val - min_val
print(f"Min : {min_val}, Max : {max_val}, Étendue : {etendue:.1f}")
```

Exercice 3 — Fonction palindrome (20 min)

Énoncé : Créer un script `palindrome.py` qui :

1. Définit `est_palindrome(mot)` — retourne True si le mot est identique dans les deux sens (ignore casse)
2. Définit `tester_phrase(phrase)` — teste chaque mot d'une phrase
3. Le programme demande une phrase et affiche quels mots sont des palindromes

Corrigé :

```
def est_palindrome(mot):
    mot = mot.lower()
    return mot == mot[::-1]

def tester_phrase(phrase):
    for mot in phrase.split():
        if est_palindrome(mot):
            print(f'"{mot}" → Oui ✓')
        else:
            print(f'"{mot}" → Non')

tester_phrase(input("Phrase : "))
```

Flux d'appel de fonction (palindrome) :

```
flowchart TD
    A[Début] --> B[Saisir phrase]
    B --> C[Appel tester_phrase]
    C --> D[Découper en mots]
    D --> E[Pour chaque mot]
    E --> F[Appel est_palindrome]
    F --> G[Convertir minuscules]
    G --> H{mot == inverse ?}
    H -->|Oui| I[Afficher ✓]
    H -->|Non| J[Afficher Non]
    I --> K[Fin boucle]
    J --> K
    K --> L[Fin]
```

Exercice 4 — Générateur d'exercices aléatoires (20 min)

Énoncé : Créer un script `generateur.py` qui génère des exercices de calcul mental pour des élèves. Le programme :

1. Définit `generer_question()` qui retourne (a, op, b, reponse) aléatoirement
2. Définit `poser_question(question)` qui pose la question et retourne True si bonne réponse
3. Propose 5 questions avec addition, soustraction, multiplication
4. Affiche la note sur 20 à la fin

Exemple attendu :

```
=== Générateur de calcul mental ===
12 + 7 = 19 ✓ Correct !
15 - 8 = 7 ✓ Correct !
6 × 9 = 54 ✓ Correct !
...
Note : 16/20
```

Corrigé :

```
import random
```

```

def generer_question():
    a = random.randint(1, 20)
    b = random.randint(1, 20)
    op = random.choice(["+", "-", "*"])
    if op == "+":
        return a, op, b, a + b
    elif op == "-":
        a, b = max(a, b), min(a, b)
        return a, op, b, a - b
    else:
        a, b = random.randint(2, 9), random.randint(2, 9)
        return a, op, b, a * b

def poser_question(a, op, b):
    try:
        return int(input(f"{a} {op} {b} = "))
    except ValueError:
        print("Réponse invalide.")
        return None

score = 0
for _ in range(5):
    a, op, b, correct = generer_question()
    rep = poser_question(a, op, b)
    if rep == correct:
        print("✓ Correct !")
        score += 1
    else:
        print(f"× Faux. Réponse : {correct}")

print(f"\nNote : {score * 4}/20")

```

Exercice 5 — Module de calculs statistiques (10 min)

Énoncé : Créer un fichier `stats_utils.py` contenant un module avec les fonctions :

1. `moyenne(liste)` — calcule la moyenne
2. `mediane(liste)` — calcule la médiane
3. `ecart_type(liste)` — calcule l'écart-type
4. `resume_statistique(liste)` — retourne toutes les stats en une fois

Créer ensuite un script `test_stats.py` qui importe le module et teste avec une liste de notes.

Corrigé — `stats_utils.py` :

```

def moyenne(liste):
    if not liste:
        return 0
    return sum(liste) / len(liste)

def mediane(liste):
    triee = sorted(liste)
    n = len(triee)
    if n % 2 == 0:
        return (triee[n // 2 - 1] + triee[n // 2]) / 2
    return triee[n // 2]

def ecart_type(liste):
    if not liste:

```

```

return 0
m = moyenne(liste)
variance = sum((x - m) ** 2 for x in liste) / len(liste)
return variance ** 0.5

def resume_statistique(liste):
    return {"min": min(liste), "max": max(liste),
            "moyenne": moyenne(liste), "mediane": mediane(liste),
            "ecart_type": ecart_type(liste), "effectif": len(liste)}

```

Corrigé — test_stats.py :

```

import stats_utils

notes = [12, 15, 10, 18, 14, 16, 11, 13]
resume = stats_utils.resume_statistique(notes)

for cle, valeur in resume.items():
    if isinstance(valeur, float):
        print(f" {cle} : {valeur:.2f}")
    else:
        print(f" {cle} : {valeur}")

```

Diagramme de module (calculs statistiques) :

```

flowchart LR
    A["stats_utils.py"] --> B[moyenne]
    A --> C[mediane]
    A --> D[ecart_type]
    A --> E[resume_statistique]
    B --> F[test_stats.py]
    C --> F
    D --> F
    E --> F
    F --> G[Résultats]

```

QCM d'auto-évaluation

1. Quel mot-clé permet de définir une fonction en Python ?

- a) function
- b) def ✓
- c) define

2. Que retourne une fonction qui n'a pas de return explicite ?

- a) 0
- b) None ✓
- c) False

3. Comment appelle-t-on une fonction nommée calculer ?

- a) call calculer()
- b) calculer() ✓
- c) run calculer()

4. Quelle est la portée d'une variable définie à l'intérieur d'une fonction ?

- a) Globale (partout dans le programme)

- b) Locale (dans la fonction uniquement) ✓
- c) Dynamique

5. Un fichier .py contenant des fonctions importables s'appelle :

- a) Une bibliothèque
- b) Un module ✓
- c) Un script

Grille d'évaluation TP 04

Critère	Poids	Non acquis (0)	Partiel
Définition de fonctions (<code>def</code>)	20 %	Ne définit pas de fonctions	Syntaxe
Paramètres et <code>return</code>	20 %	N'utilise pas de paramètres	Utilisatio
Appels et réutilisation	20 %	N'appelle pas les fonctions	Appels e
Docstrings et documentation	15 %	Aucune docstring	Docstrin
Module et import	15 %	Pas de module	Import in
Structuration du code	10 %	Code non structuré	Structur

Déroulement séance — 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil et rappel fonctions	10 min	10 min
Exercice 1 — calculer_moyenne	15 min	25 min
Exercice 2 — max/min personnalisés	15 min	40 min
Exercice 3 — Palindrome	20 min	60 min
Exercice 4 — Générateur d'exercices	20 min	80 min
Exercice 5 — Module statistique	10 min	90 min

Note pédagogique ENS

Ce TP conclusif sur les fonctions et la modularité montre comment l'abstraction (encapsuler du code) permet de construire des programmes réutilisables. Le générateur d'exercices (ex. 4) illustre comment Python crée du matériel pédagogique interactif pour le primaire et le collège. Le module statistique (ex. 5) prépare les étudiants à créer leurs propres outils d'analyse de notes. L'enseignant peut insister sur l'importance des docstrings et inviter les étudiants à adapter ces exercices pour différents niveaux scolaires (cycle 3, cycle 4, lycée), développant ainsi la différenciation pédagogique.

Prolongement possible : Créer un module complet de gestion de notes avec import/export CSV, coefficients, et génération d'un bulletin PDF (via reportlab ou fpdf).